

TP 1 - Introduction à MATLAB

Ali ZAINOUL
ali.zainoul.az@gmail.com

13 novembre 2025

1. Fonctions d’affichage : disp, fprintf et sprintf

MATLAB propose plusieurs fonctions pour afficher des informations dans la console. Comprendre ces fonctions et leurs options de formatage est essentiel pour produire une sortie claire et lisible.

disp

La fonction `disp` est utilisée pour afficher simplement du texte ou des variables sans mise en forme particulière :

```
1 disp('hello_world!');  
2 disp(3.1415);
```

fprintf

La fonction `fprintf` permet d’afficher une sortie formatée dans la fenêtre de commande.

```
1 fprintf('the_value_of_pi_is_approximately: %.2f\n', 3.1415);
```

sprintf

```
1 formatted_str = sprintf('pi_to_two_decimals_is: %.2f', 3.1415);  
2 disp(formatted_str);
```

Spécificateur	Type	Exemple
%d	Entier	fprintf("integer: %d", 10);
%f	Réel	fprintf("float: %.2f", 3.1415);
%s	Chaîne	fprintf("string: %s", "matlab");

TABLE 1 – Spécificateurs de format courants

Spécificateurs de format

2. Fonctions d'entrée utilisateur

input

```
1 user_age = input('enter_your_age: ');
2 user_name = input('enter_your_name: ', 's');
```

menu

```
1 user_choice = menu('choose_a_color', 'red', 'blue', 'green');
```

3. Scripts et fonctions

3.1 Scripts

```
1 % myscript.m
2 x = 5;
3 y = 3;
4 disp(x + y);
```

3.2 Fonctions

```
1 function output_val = square_number(input_val)
2     output_val = input_val ^ 2;
3 end
```

4. Structures conditionnelles

Les structures conditionnelles permettent d'exécuter du code selon une condition logique.

4.1 Structure if-else

```
1 x = input('enter_a_number: ');
2 if mod(x,2) == 0
3     disp('the_number_is_even');
4 else
5     disp('the_number_is_odd');
6 end
```

4.2 Structure elseif

```
1 score = input('enter_score: ');
2 if score >= 90
3     disp('grade_A');
4 elseif score >= 75
5     disp('grade_B');
6 else
7     disp('grade_C');
8 end
```

5. Boucles

Les boucles permettent de répéter une série d'instructions plusieurs fois.

5.1 Boucle for

```
1 for i = 1:5
2     fprintf('iteration_number_%d\n', i);
3 end
```

5.2 Boucle while

```
1 count = 1;
2 while count <= 5
3     fprintf('count_%d\n', count);
4     count = count + 1;
5 end
```

6. Tableaux et matrices

Les tableaux sont la base de MATLAB (Matrix Laboratory).

6.1 Création

```
1 A = [1 2 3 4];
2 B = [1; 2; 3; 4];
```

6.2 Opérations de base

```
1 sum_A = sum(A);
2 mean_A = mean(A);
3 std_A = std(A);
```

6.3 Tableaux aléatoires

```
1 R = rand(1,5); % random numbers between 0 and 1
```

7. Tableaux de cellules

Les cellules permettent de stocker des éléments de types variés.

```
1 C = {'Alice', 25, [90 85 88]};
2 fprintf('name: %s\n', C{1});
```

8. Structures

Les structures regroupent des données hétérogènes dans des champs nommés.

```
1 student.name = 'Alice';
2 student.age = 21;
3 student.grades = [12 15 14];
4 student.average = mean(student.grades);
5 fprintf('student: %s, average: %.2f\n', student.name, student.average);
```

9. Exercices

Exercice 1 : Utiliser disp et fprintf

Écrivez un script qui affiche les éléments suivants :

- Un message **Learning MATLAB Formatting!** à l'aide de `disp`.
- La racine carrée de 10 avec deux décimales à l'aide de `fprintf`.

Exercice 2 : Créer un menu simple

Écrivez un script qui utilise la fonction `menu` pour demander à l'utilisateur de sélectionner son langage de programmation préféré (par exemple : MATLAB, Python, Java).

Exercice 3 : Écrire une fonction

Créez une fonction `celsius_to_fahrenheit` qui convertit une température de degrés Celsius en degrés Fahrenheit. La formule de conversion est :

$$F = \frac{9}{5}C + 32$$

Testez la fonction à partir d'un script avec des valeurs d'exemple.

Exercice 4 : Utiliser `sprintf` pour formater une chaîne

Écrivez un script qui calcule l'aire d'un cercle de rayon 5 et formate le résultat dans une chaîne à l'aide de `sprintf`.

Exercice 5 : Utiliser des boucles et des conditions

Écrivez un script qui :

- Demande à l'utilisateur d'entrer un entier positif.
- Affiche tous les nombres pairs inférieurs ou égaux à ce nombre à l'aide d'une boucle `for`.
- Utilise une structure `if` pour vérifier si le nombre est pair ou impair.

Exercice 6 : Créer un tableau et afficher ses statistiques

Écrivez un script qui :

- Crée un vecteur de nombres aléatoires à l'aide de `rand`.
- Calcule la moyenne, l'écart-type et la somme.
- Affiche ces valeurs à l'aide de `fprintf` avec un formatage clair.

Exercice 7 : Lecture d'entrées multiples

Créez un script qui demande à l'utilisateur d'entrer trois valeurs numériques, puis affiche :

- Leur somme.
- Leur moyenne.
- Le maximum et le minimum.

Exercice 8 : Créer une fonction personnalisée

Créez une fonction `stats_vector` qui prend un vecteur en entrée et retourne :

- sa longueur ;
- sa somme ;
- sa moyenne ;
- son écart-type.

Affichez les résultats dans un script principal à l'aide de `fprintf` et d'une chaîne formatée `fmt`.

Exercice 9 : Affichage de texte formaté avec `fmt`

Reprenez l'exercice précédent et modifiez le script principal pour utiliser une seule chaîne `fmt` contenant plusieurs lignes. Affichez les résultats des trois premiers vecteurs à l'aide d'une boucle et de `fprintf(fmt, ...)`.

Exercice 10 : Chaînes de caractères et tableaux de cellules

Écrivez un script qui :

- Crée un tableau de cellules contenant plusieurs prénoms.
- Parcourt chaque élément à l'aide d'une boucle `for`.
- Utilise `fprintf` pour afficher chaque prénom avec son index.

Exercice 11 : Combiner `sprintf` et `disp`

Créez un script qui :

- Utilise `sprintf` pour formater un message personnalisé.
- Stocke ce message dans une variable `msg`.
- Affiche `msg` à l'aide de `disp`.

Exercice 12 : Manipulation de structures

Écrivez un script qui :

- Crée une structure `student` avec les champs `name`, `age`, et `grades`.
 - Calcule la moyenne des notes et l'ajoute à la structure sous le champ `average`.
 - Affiche le contenu complet de la structure avec `fprintf` et `sprintf`.
-

Corrections des exercices

Correction Exercice 1 : Affichage formaté avec gestion d'erreur

```
1 try
2     disp('Learning MATLAB Formatting!');
3     fprintf('Square root of 10 = %.2f\n', sqrt(10));
4 catch err
5     fprintf('Error: %s\n', err.message);
6 end
```

Correction Exercice 2 : Menu et gestion d'indice

```
1 try
2     choice = menu('Select your favorite programming language:', ...
3                 'MATLAB', 'Python', 'Java');
4
5     if choice == 0
6         disp('No selection made.');
```

```
7     else
8         options = ['MATLAB', 'Python', 'Java'];
9         fprintf('You selected option %d: %s\n', choice, options(choice));
10    end
11 catch err
12    fprintf('Error: %s\n', err.message);
13 end
```

Correction Exercice 3 : Conversion Celsius–Fahrenheit

```
1 function f = celsius_to_fahrenheit(c)
2     try
3         f = (9/5) * c + 32;
4     catch err
5         fprintf('Error in conversion: %s\n', err.message);
6         f = NaN;
7     end
8 end
9
10 % Example test
11 try
12     temperature_c = 25;
13     temperature_f = celsius_to_fahrenheit(temperature_c);
14     fprintf('%.2f C = %.2f F\n', temperature_c, temperature_f);
15 catch err
16     fprintf('Error: %s\n', err.message);
17 end
```

Correction Exercice 4 : Aire d'un cercle

```
1 try
2     r = 5;
3     area = pi * r^2;
4     res = sprintf('The area of a circle with radius %d is %.2f\n', r, area);
5     disp(res)
6 catch err
7     fprintf('Error: %s\n', err.message);
8 end
```

Correction Exercice 5 : Nombres pairs

```
1  %{
2      Script Name: even_odd_analysis.m
3      Description:
4          This script asks the user to input a positive integer,
5          validates the input, and then iterates through all integers
6          from 1 up to the given number. For each number, it determines
7          whether it is even or odd using the mod() function and prints
8          the result to the console.
9
10     Key Features:
11         - Robust input validation
12         - Structured error handling with try/catch
13         - Clear parity check within a loop
14         - Uses fprintf for formatted, readable output
15     %}
16
17     try
18         % Ask the user to enter a positive integer
19         user_input = input('Enter a positive integer: ');
20
21         %{
22             Input validation:
23             - isempty(user_input): ensures the user actually entered something.
24             - isnumeric(user_input): ensures the input is numeric.
25             - user_input <= 0: ensures the number is strictly positive.
26             - mod(user_input, 1) ~= 0: ensures the number is an integer
27               (no decimals allowed).
28         %}
29         if isempty(user_input) || ~isnumeric(user_input) || user_input <= 0 || mod(user_input,1) ~= 0
30             error('The entered value must be a positive integer.');
```

Correction Exercice 6 : Statistiques d'un vecteur aléatoire

```
1  try
2      v = rand(1,10);
3      fprintf('Vector mean = %.2f, std = %.2f, sum = %.2f\n', ...
4          mean(v), std(v), sum(v));
5  catch err
6      fprintf('Error: %s\n', err.message);
7  end
```

Correction Exercice 7 : Saisie multiple et résumé


```

1 try
2     x = input('Enter first number: ');
3     y = input('Enter second number: ');
4     z = input('Enter third number: ');
5
6     nums = [x, y, z];
7     fprintf('Sum = %.2f, Mean = %.2f\n', sum(nums), mean(nums));
8     fprintf('Max = %.2f, Min = %.2f\n', max(nums), min(nums));
9 catch err
10    fprintf('Error: %s\n', err.message);
11 end

```

Correction Exercice 8 : Fonction statistiques avec sortie multiple

```

1 function [len_, sum_, avg_, std_] = stats_vector(v)
2     try
3         len_ = length(v);
4         sum_ = sum(v);
5         avg_ = mean(v);
6         std_ = std(v);
7     catch err
8         fprintf('Error in stats_vector: %s\n', err.message);
9         [len_, sum_, avg_, std_] = deal(NaN);
10    end
11 end
12
13 % Main script
14 try
15     my_vector = [1, 2, 3, 4];
16     [len_, sum_, avg_, std_] = stats_vector(my_vector);
17     fmt = 'Length: %d \nSum: %.2f \nMean: %.2f \nStd: %.2f \n';
18     fprintf(fmt, len_, sum_, avg_, std_);
19 catch err
20     fprintf('Error: %s\n', err.message);
21 end

```

Correction Exercice 9 : Boucle sur plusieurs vecteurs

```

1 try
2     my_vectors = {[1 2 3 4], [2.0 5.2 4.8], [-2.2 -1.2 -9.6 10.5]};
3     for index = 1:numel(my_vectors)
4         [len_, sum_, avg_, std_] = stats_vector(my_vectors{index});
5         fmt = '\n --- Test Case %d --- \n Length: %d \n Sum: %.2f \n Mean: %.2f \n Std: %.2f \n';
6         fprintf(fmt, index, len_, sum_, avg_, std_);
7     end
8 catch err
9     fprintf('Error: %s\n', err.message);
10 end

```

Correction Exercice 10 : Cellule de noms et indice

```

1 try
2     names = {'Alice', 'Bob', 'Charlie', 'Diana'};
3     for i = 1:numel(names)
4         fprintf('Name %d: %s\n', i, names{i});
5     end
6 catch err
7     fprintf('Error: %s\n', err.message);
8 end

```

Correction Exercice 11 : Combinaison de sprintf et disp

```
1 try
2     name = 'Alice';
3     score = 95;
4     msg = sprintf('Student %s scored %d points.', name, score);
5     disp(msg);
6 catch err
7     fprintf('Error: %s\n', err.message);
8 end
```

Correction Exercice 12 : Manipulation d'une structure

```
1 try
2     student.name = 'Alice';
3     student.age = 21;
4     student.grades = [14, 16, 18];
5     student.average = mean(student.grades);
6
7     fprintf('Name: %s\nAge: %d\nAverage grade: %.2f\n', ...
8             student.name, student.age, student.average);
9     disp(student);
10 catch err
11     fprintf('Error: %s\n', err.message);
12 end
```